

Data Preparation

1 Handling Missing Values

Why Missing Values Exist

- Data entry errors
- Survey questions skipped
- Sensor failures
- Data corruption during collection

Types of Missing Data

MCAR — Missing Completely At Random: No pattern. Any row can have missing value. Safe to drop.

MAR — Missing At Random: Missing values depend on other features. Need careful handling.

MNAR — Missing Not At Random: Missing values have a pattern. Most dangerous — can bias the model.

How to Handle Missing Values

1 — Drop rows: Remove rows with missing values. Only use when very few rows are missing — less than 5%.

Advantage — simple and clean

Disadvantage — lose data

2 — Drop columns: Remove entire column if more than 50-60% values are missing.

Advantage — removes useless feature

Disadvantage — might lose important information

3 — Mean Imputation: Replace missing value with mean of that column. Use for numerical data with no outliers.

Advantage — simple

Disadvantage — reduces variance, ignores relationships

4 — Median Imputation: Replace missing value with median. Use for numerical data with outliers. More robust than mean.

Advantage — not affected by outliers

Disadvantage — ignores relationships between features

5 — Mode Imputation: Replace missing value with most frequent value. Use for categorical data.

Advantage — works for categories

Disadvantage — can create imbalance

6 — Forward Fill / Backward Fill: Fill missing value with previous or next value. Used for time series data.

7 — KNN Imputation: Use K nearest neighbors to fill missing values. More accurate but slower.

8 — Model Based Imputation: Train a model to predict missing values using other features. Most accurate but complex.

Rule of Thumb

Missing %	Action
Less than 5%	Drop rows
5% to 30%	Impute — mean, median, mode
30% to 60%	Impute carefully or create indicator column
More than 60%	Drop the column

2 Feature Scaling

Why Feature Scaling is Needed

Different features have different ranges:

- Age — 0 to 100
- Salary — 10000 to 1000000
- Blood pressure — 60 to 200

Without scaling — model gives more importance to features with larger values even if they are not more important. This is mathematically wrong.

Two Main Types

Normalization — Min Max Scaling:

Scales all values between 0 and 1.

$$X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Example:

Age values: [20, 40, 60, 80]

Min = 20, Max = 80

20 → 0.0

40 → 0.33

60 → 0.67

80 → 1.0

When to use:

- When you know the minimum and maximum values
- When data does not have many outliers
- Neural networks — need values between 0 and 1
- Image data — pixel values 0 to 255 scaled to 0 to 1

Weakness:

- Sensitive to outliers — one extreme value compresses all others
-

Standardization — Standard Scaler:

Scales data to have mean of 0 and standard deviation of 1.

$X_{\text{scaled}} = (X - \text{mean}) / \text{standard deviation}$

Example:

Age values: [20, 40, 60, 80]

Mean = 50, Std = 22.36

20 → -1.34

40 → -0.45

60 → +0.45

80 → +1.34

When to use:

- When data has outliers
- Logistic Regression, SVM, KNN, PCA — these need standardization
- When you don't know the range of data
- Most common choice in ML projects

Weakness:

- Does not bound values between 0 and 1
 - Assumes data is normally distributed
-

Which Algorithms Need Scaling

Algorithm	Needs Scaling?
Linear Regression	Yes
Logistic Regression	Yes
KNN	Yes — distance based
SVM	Yes — distance based
PCA	Yes
Decision Tree	No — based on splits
Random Forest	No
XGBoost	No
Naive Bayes	No

Rule: Distance based algorithms always need scaling. Tree based algorithms do not.

3 Handling Imbalanced Data

What is Imbalanced Data?

When one class has significantly more samples than another.

Example — Heart disease dataset:

No disease — 900 samples — 90%

Disease — 100 samples — 10%

This is a problem because the model will be biased towards the majority class. It can predict "no disease" for everyone and still get 90% accuracy — but this is useless and dangerous.

Techniques to Handle Imbalance

1 — Oversampling — SMOTE: Synthetic Minority Oversampling Technique. Creates synthetic samples for minority class.

Before SMOTE: No disease 900, Disease 100

After SMOTE: No disease 900, Disease 900

Most popular and effective technique.

2 — Undersampling: Reduce majority class samples to match minority class.

Before: No disease 900, Disease 100

After: No disease 100, Disease 100

Simple but loses a lot of data.

3 — Class Weight: Tell the model to give more importance to minority class during training. Most algorithms in sklearn have `class_weight="balanced"` parameter.

4 — Use Right Metric: Do not use accuracy for imbalanced data. Use:

- Precision, Recall, F1 Score
- ROC AUC Score
- Confusion Matrix

Rule of Thumb

Imbalance Ratio Technique

70/30	Use class weights
80/20	SMOTE or class weights
90/10 or worse	SMOTE + right evaluation metric

Feature Engineering & Feature Selection

What is Feature Engineering?

Creating new features from existing ones to help the model learn better.

Examples:

From date column:

date → year, month, day, day_of_week, is_weekend

From name column:

full_name → first_name, last_name, name_length

From numerical columns:

height + weight → BMI

salary + expenses → savings_rate

Combining features:

rooms × floor_area → total_space

What is Feature Selection?

Choosing only the most important features and removing irrelevant or redundant ones.

Why it matters:

- Too many features causes overfitting
- Irrelevant features add noise
- Fewer features means faster training
- Improves model interpretability

Feature Selection Techniques

1 — Filter Methods: Select features based on statistical relationship with target.

- Correlation — remove highly correlated features
- Chi-square test — for categorical features
- ANOVA — for numerical features

2 — Wrapper Methods: Train model with different feature subsets and pick best.

- Recursive Feature Elimination — RFE
- Forward selection — add features one by one
- Backward elimination — remove features one by one

3 — Embedded Methods: Feature selection happens during model training.

- Lasso — reduces unimportant feature weights to zero
- Random Forest feature importance — ranks features by importance
- XGBoost feature importance

Feature Importance — Random Forest Example

After training Random Forest you can check which features contributed most to predictions. This tells you which features are truly important for your problem.

For heart disease — you might find:

age — most important

cholesterol — second most important

blood pressure — third

resting heart rate — fourth

sex — less important



Practice These

How do you handle missing values?

"First I check the percentage of missing values per column. If less than 5% I drop those rows. If a column has more than 60% missing I drop the column. For numerical columns I use median imputation because it is robust to outliers. For categorical columns I use mode imputation. For time series data I use forward fill. In some cases I use KNN imputation for more accurate results."

What is the difference between normalization and standardization?

"Normalization scales values between 0 and 1 using min-max formula. It is sensitive to outliers. Standardization scales data to have mean 0 and standard deviation 1. It is more robust to outliers. I use standardization for most ML algorithms especially distance based ones like KNN and SVM. Normalization is preferred for neural networks and image data."

How do you handle imbalanced data?

"For imbalanced datasets I use SMOTE to oversample the minority class and create synthetic samples. I also use class_weight balanced parameter in sklearn. Most importantly I avoid using accuracy as evaluation metric and instead use precision, recall, F1 score and ROC AUC score which give a true picture of model performance on imbalanced data."

What is the difference between feature engineering and feature selection?

"Feature engineering is creating new features from existing ones to help the model learn better patterns. Feature selection is choosing the most important features and removing irrelevant ones to reduce overfitting and improve performance. Both are important steps in the ML workflow."